

SDM

Version Control with Git (Team-Based Development)

Q1. The primary purpose of a Version Control System (VCS) is to:

- A) Run code faster
- B) Track and manage changes to software over time
- C) Replace testing tools
- D) Compile source code

✅ **Answer: B**

Explanation: VCS stores change history, supports collaboration and rollback.

Q2. Without version control, the most common issue in team development is:

- A) Better performance
- B) Easy dependency handling
- C) Files getting overwritten and no history tracking
- D) Automatic merging always works

✅ **Answer: C**

Explanation: Manual sharing causes overwrites, conflicts, and no traceability.

Q3. "Traceability" in team-based development means:

- A) Faster execution
- B) Ability to track who changed what and when
- C) Automatic bug fixing
- D) No need for backups

✅ **Answer: B**

Explanation: VCS maintains author + time + change history.

Q4. Which of the following is a major benefit of using VCS?

- A) Code becomes unreadable
- B) Cannot revert changes
- C) Reversible history and backup of code
- D) Removes merge conflicts permanently

✅ **Answer: C**

Explanation: VCS provides rollback + history + recovery.

Q5. In a Local VCS, versions are stored:

- A) Only on a central server
- B) On one computer locally
- C) Only in cloud storage
- D) Only in remote repository

✅ **Answer: B**

Explanation: Local VCS stores history on the same machine.

Q6. Which is an example of a Centralized VCS?

- A) Git
- B) Mercurial
- C) Subversion (SVN)
- D) Bitbucket

☒ **Answer: C**

Explanation: SVN is centralized; Git is distributed.

Q7. Distributed Version Control System means:

- A) Only one shared repository exists
- B) Every developer has a full copy of repository history
- C) Developers cannot work offline
- D) No branching support exists

☒ **Answer: B**

Explanation: Git provides full local repo to each developer.

Q8. Which of the following is a Distributed VCS?

- A) SVN
- B) CVS
- C) Git
- D) RCS

☒ **Answer: C**

Explanation: Git is the most popular DVCS.

Q9. Git is considered industry standard mainly because it is:

- A) Centralized and slow
- B) Distributed, fast, secure, supports branching/merging
- C) Only works online
- D) Has no history feature

☒ **Answer: B**

Explanation: Git offers local full history + powerful branching.

Q10. In Git, developers can work offline because:

- A) Git does not use commits
- B) Remote repo is mandatory for all operations
- C) Full repository history exists locally
- D) Git stores code only in cloud

☒ **Answer: C**

Explanation: Most Git operations work without internet.

Q11. Which Git area contains the files currently being worked on?

- A) Remote repository
- B) Local repository
- C) Working directory
- D) Staging area

✅ **Answer: C**

Explanation: Working directory = current project files.

Q12. Staging area (Index) is used for:

- A) Running programs
- B) Preparing files before commit
- C) Hosting repository online
- D) Deleting old commits

✅ **Answer: B**

Explanation: git add puts changes into staging area.

Q13. Correct Git workflow is:

- A) Working → Local Repo → Staging → Remote
- B) Working → Staging → Local Repo → Remote
- C) Local Repo → Working → Remote → Staging
- D) Staging → Working → Remote → Local

✅ **Answer: B**

Explanation: This is the standard Git flow.

Q14. A commit in Git represents:

- A) A temporary file
- B) A version snapshot of the project
- C) A deleted history
- D) A cloud backup only

✅ **Answer: B**

Explanation: Commit stores changes permanently in local repo history.

Q15. Which command initializes a new Git repository in the current folder?

- A) git start
- B) git init
- C) git begin
- D) git new

✅ **Answer: B**

Explanation: git init creates local repo metadata.

Q16. Which command is used to create a local copy of an existing remote repository?

- A) git pull
- B) git clone
- C) git push
- D) git fork

✅ **Answer: B**

Explanation: git clone <url> downloads entire repo history.

Q17. Which command shows current repository state (modified/staged/untracked files)?

- A) git show
- B) git status
- C) git state
- D) git list

✅ **Answer: B**

Explanation: git status displays file state summary.

Q18. Which command stages all changes in the current directory?

- A) git commit -m "msg"
- B) git add *
- C) git add .
- D) git push

✅ **Answer: C**

Explanation: git add . stages all changed/untracked files.

Q19. Which command creates a commit with a message?

- A) git commit -m "message"
- B) git add -m "message"
- C) git push -m "message"
- D) git init -m "message"

✅ **Answer: A**

Explanation: git commit -m saves staged snapshot in local history.

Q20. Which command shows commit history?

- A) git status
- B) git log
- C) git diff
- D) git branch

✅ **Answer: B**

Explanation: git log prints commit list.

Q21. git push is used to:

- A) Download remote changes
- B) Upload local commits to remote repository
- C) Delete remote repository
- D) Create branch locally

✅ **Answer: B**

Explanation: Push sends commits from local repo → remote repo.

Q22. git pull is used to:

- A) Upload local changes
- B) Download + integrate remote changes
- C) Delete local repository
- D) Only compare commits

✅ **Answer: B**

Explanation: Pull = fetch + merge (commonly).

Q23. A branch in Git is:

- A) A backup file
- B) A parallel line of development
- C) A repository clone
- D) A commit message format

✅ **Answer: B**

Explanation: Branch enables working on features independently.

Q24. Command to create a new branch named feature1 is:

- A) git checkout feature1
- B) git branch feature1
- C) git merge feature1
- D) git add feature1

✅ **Answer: B**

Explanation: git branch feature1 creates branch (does not switch).

Q25. Command to switch to branch feature1 is:

- A) git switch feature1
- B) git checkout feature1
- C) git branch feature1
- D) git pull feature1

✅ **Answer: B**

Explanation: Classic command: git checkout <branch>.

Q26. Command to create and switch branch in one step:

- A) git checkout -b feature1
- B) git branch -b feature1
- C) git init -b feature1
- D) git merge -b feature1

✅ **Answer: A**

Explanation: -b means create new branch + checkout.

Merging

Q27. To merge branch feature1 into main, correct steps are:

- A) checkout feature1 → merge main
- B) checkout main → merge feature1
- C) checkout main → add feature1
- D) checkout feature1 → pull main

✅ **Answer: B**

Explanation: Always merge *into* the target branch after checking it out.

Q28. Command to delete a local branch after merge:

- A) git branch -d feature1
- B) git delete feature1
- C) git rm feature1
- D) git cut feature1

✅ **Answer: A**

Explanation: -d deletes safely (if merged).

Q29. git fetch does:

- A) Fetch + merge automatically
- B) Fetches remote changes but does NOT merge
- C) Deletes remote commits
- D) Pushes local changes

✅ **Answer: B**

Explanation: Fetch only downloads updates, keeps your branch unchanged.

Q30. git pull is equivalent to:

- A) git fetch + git merge
- B) git add + git commit
- C) git commit + git push
- D) git merge + git push

✅ **Answer: A**

Explanation: Pull usually performs fetch then merge.

Q31. git diff is used to:

- A) Delete files
- B) Show differences between changes
- C) Push commits
- D) Create branches

✅ **Answer: B**

Explanation: Shows changes between versions/staged/working.

Q32. A merge conflict occurs when:

- A) Only one developer works
- B) Two developers modify the same block of code
- C) You run git status
- D) You delete a file manually

✅ **Answer: B**

Explanation: Git cannot auto-merge conflicting edits.

Q33. After resolving merge conflict manually, the correct next steps are:

- A) git log → git fetch
- B) git add → git commit
- C) git push → git status
- D) git clone → git init

✅ **Answer: B**

Explanation: Resolution must be staged and committed.

Q34. Correct industry workflow is:

- A) Work on main directly → push → deploy
- B) Create branch → commit → push → PR → review → merge
- C) Delete branch first → commit later
- D) Merge first → write code later

✅ **Answer: B**

Explanation: Feature branch + PR + review is standard.

Q35. Pull Request (PR) is mainly used for:

- A) Deleting branches automatically
- B) Code review and controlled merge into main
- C) Compressing code
- D) Increasing RAM

✅ **Answer: B**

Explanation: PR ensures quality and traceability.



Software Engineering Foundations (SDM)

Q1. Software consists of:

- A) Only programs (code)
- B) Only documentation
- C) Programs + execution-time data + configuration + documentation
- D) Only configuration files

✓ **Answer: C**

Explanation: Software is not just code; it includes data, configs, and docs.

Q2. Which is an example of “Software as a Product”?

- A) Compiler
- B) IDE
- C) Banking System Application
- D) Testing Tool

✓ **Answer: C**

Explanation: Product software is directly used by end users.

Q3. Which is an example of “Software as a Development Environment/Tool”?

- A) ERP system
- B) Online banking portal
- C) Compiler
- D) Inventory management software

✓ **Answer: C**

Explanation: Compiler is used to build other software.

Q4. Hardware is generally:

- A) Engineered and does not wear out
- B) Manufactured and wears out physically
- C) Not physical at all
- D) Always custom-built

✓ **Answer: B**

Explanation: Hardware fails due to physical degradation.

Q5. The correct statement about software “wear out” is:

- A) Software wears out like hardware
- B) Software fails due to physical stress
- C) Software does not wear out physically, it deteriorates due to changes
- D) Software becomes useless after 5 years always

✓ **Answer: C**

Explanation: Software degrades due to modifications, complexity, bugs.

Q6. Software is primarily:

- A) Manufactured on an assembly line
- B) Engineered and developed

- C) Only purchased and installed
- D) Physically produced

✓ **Answer: B**

Explanation: Software is engineered, not manufactured.

Q7. Which is an example of System Software?

- A) MS Word
- B) Operating System
- C) Banking system
- D) E-commerce website

✓ **Answer: B**

Explanation: OS/compilers/file managers are system software.

Q8. Software built into hardware devices is called:

- A) Web software
- B) Embedded software
- C) AI software
- D) Product-line software

✓ **Answer: B**

Explanation: Embedded software runs inside devices (router, washing machine).

Q9. “Web Applications” are typically:

- A) Only desktop apps
- B) Executed over the internet
- C) Only embedded systems
- D) Only compilers

✓ **Answer: B**

Explanation: Web apps run via network/browser environment.

Q10. AI Software mainly focuses on:

- A) Numerical-only calculations
- B) File system management
- C) Non-numerical reasoning and decision logic
- D) Hardware manufacturing

✓ **Answer: C**

Explanation: AI handles complex logic (expert systems, ML, robotics).

Q11. Software Engineering is:

- A) Writing code without planning
- B) Systematic, disciplined, measurable approach to software development
- C) Only testing activity
- D) Only documentation preparation

✓ **Answer: B**

Explanation: SE applies engineering discipline to development + maintenance.

Q12. Software Engineering is needed mainly to ensure software is:

- A) Only fast
- B) Reliable, maintainable, cost-effective, delivered on time
- C) Always free
- D) Always open-source

☒ **Answer: B**

Explanation: These are core SE goals.

Q13. “Software Crisis” was formally recognized in:

- A) 1941
- B) 1950
- C) 1968 (NATO Conference)
- D) 2000

☒ **Answer: C**

Explanation: 1968 NATO introduced the term “Software Engineering”.

Q14. One key reason behind Software Crisis was:

- A) Too much RAM
- B) Increasing complexity + weak project control/testing
- C) Too many programming languages
- D) Too many CPUs

☒ **Answer: B**

Explanation: Complexity, poor methodology, and unrealistic schedules caused it.

Q15. Building a completely new software system from scratch is:

- A) Production support
- B) Full development project
- C) Conversion project
- D) Regular maintenance

☒ **Answer: B**

Explanation: Full lifecycle activities are included.

Q16. Developing only UI module for an existing system is:

- A) Full development
- B) Part development
- C) Migration
- D) Porting

☒ **Answer: B**

Explanation: Part development = only partial lifecycle/module.

Q17. Bug fixing and adapting software for updated OS is:

- A) Enhancement
- B) Regular maintenance
- C) Porting
- D) Re-engineering

✅ **Answer: B**

Explanation: Maintenance = corrective/adaptive/extension activity.

Q18. Adding a new major module like “Payment Gateway Integration” is:

- A) Regular maintenance
- B) Enhancement
- C) Production support
- D) Orphan process

✅ **Answer: B**

Explanation: Enhancement adds significant value/features.

Q19. Monitoring uptime and handling incidents in real-time is:

- A) Production support
- B) Porting
- C) Full development
- D) Requirement elicitation

✅ **Answer: A**

Explanation: Production support ensures smooth running + SLA handling.

Q20. Converting COBOL system into Java while keeping same business logic is:

- A) Porting
- B) Migration
- C) Re-engineering
- D) Prototyping

✅ **Answer: C**

Explanation: Re-engineering improves structure/tech but keeps functionality.

Q21. Moving application from on-premise to cloud with minimal code change is:

- A) Migration
- B) Porting
- C) Re-engineering
- D) Enhancement

✅ **Answer: A**

Explanation: Migration = environment move (cloud/db/server).

Q22. Making Windows app work on Linux is:

- A) Migration
- B) Porting
- C) Reuse management
- D) Maintenance

✅ **Answer: B**

Explanation: Porting = multi-platform enablement.

Q23. A software process defines:

- A) Only coding rules
- B) Who does what, when, and how during development
- C) Only testing schedule
- D) Only UI design

✅ **Answer: B**

Explanation: Process organizes and controls development steps.

Q24. Which is NOT a core framework activity?

- A) Communication
- B) Planning
- C) Modeling
- D) Decompilation

✅ **Answer: D**

Explanation: Generic activities are Communication, Planning, Modeling, Construction, Deployment.

Q25. Coding + Testing comes under:

- A) Construction
- B) Deployment
- C) Modeling
- D) Communication

✅ **Answer: A**

Explanation: Construction includes building and verifying code.

Q26. Risk management and quality assurance are examples of:

- A) Core process activities
- B) Umbrella activities
- C) Hardware processes
- D) Database tuning

✅ **Answer: B**

Explanation: Umbrella activities support all phases.

Q27. SDLC means:

- A) Software Design Level Coding
- B) Software Development Life Cycle
- C) System Design Logic Controller
- D) Software Documentation Listing Chart

✅ **Answer: B**

Explanation: SDLC covers requirement → maintenance lifecycle.

Q28. Waterfall model is:

- A) Iterative and flexible
- B) Linear sequential model
- C) Always incremental
- D) Used only in Agile

✓ **Answer: B**

Explanation: Each phase finishes before next begins.

Q29. Waterfall is best suited when:

- A) Requirements change daily
- B) Requirements are stable and clear
- C) No documentation allowed
- D) Team is unknown

✓ **Answer: B**

Explanation: Waterfall works best with stable requirements.

Q30. A major weakness of Waterfall is:

- A) Too much feedback
- B) Late customer feedback and high rigidity
- C) No structure
- D) No testing possible

✓ **Answer: B**

Explanation: Working product comes late, changes are costly.

Q31. Incremental model delivers software:

- A) Only once at the end
- B) In small usable increments
- C) Only as prototype
- D) Only as documentation

✓ **Answer: B**

Explanation: Each increment is a working version release.

Q32. Incremental model is best suited when:

- A) Requirements never change
- B) Requirements evolve and time-to-market is important
- C) System is extremely small
- D) No integration is required

✓ **Answer: B**

Explanation: It supports change and early delivery.

Q33. RAD emphasizes:

- A) Long documentation
- B) Prototyping + CASE tools + user workshops
- C) Only maintenance
- D) Only risk analysis

✓ **Answer: B**

Explanation: RAD accelerates development via rapid prototyping.

Q34. Spiral model is best known for:

- A) Removing risk analysis
- B) Strong risk analysis in every iteration
- C) No customer involvement
- D) Only linear phases

✅ **Answer: B**

Explanation: Spiral combines iteration + risk evaluation.

Q35. V-Model is mainly used to:

- A) Remove testing phases
- B) Map every development phase to corresponding testing phase
- C) Avoid documentation
- D) Enable only prototyping

✅ **Answer: B**

Explanation: Requirement ↔ acceptance test, design ↔ system test, etc.

Q36. Agile development focuses on:

- A) Rigid planning and no changes
- B) Collaboration, customer feedback, and adapting to change
- C) Only architecture design
- D) No iterations

✅ **Answer: B**

Explanation: Agile welcomes changing requirements + iterative delivery.

Q37. Prescriptive models are generally:

- A) Structured, predictable, documentation-heavy
- B) Random and uncontrolled
- C) Only used in AI systems
- D) Only for prototyping

✅ **Answer: A**

Explanation: Waterfall/Incremental/RAD fit prescriptive approach.

Q38. Evolutionary models mainly support:

- A) No feedback loops
- B) Iteration and continuous refinement
- C) Only single release
- D) No requirement changes

✅ **Answer: B**

Explanation: Prototyping, Spiral, Agile are evolutionary.

Q39. Requirement describes:

- A) HOW system will be designed
- B) WHAT system must do and constraints
- C) Only code syntax
- D) Only database schema

✓ **Answer: B**

Explanation: Requirement = what, design = how.

Q40. Requirement Engineering is:

- A) Coding activity
- B) Systematic process of identifying, documenting, validating, managing requirements
- C) Only deployment activity
- D) Only testing activity

✓ **Answer: B**

Explanation: RE covers elicitation → validation → management.

Q41. A good requirement must be:

- A) Ambiguous
- B) Implementation-free, verifiable, complete, consistent
- C) Only written in code
- D) Only based on developer assumptions

✓ **Answer: B**

Explanation: Good requirements should be testable and unambiguous.

Q42. Functional requirements define:

- A) Quality attributes like performance
- B) Services/behavior system must provide
- C) Project deadlines
- D) Hardware cost

✓ **Answer: B**

Explanation: Functional = what system does.



Q43. Non-functional requirements define:

- A) Services like login validation
- B) Quality constraints like security, reliability, performance
- C) Database table names only
- D) Code indentation rules

✓ **Answer: B**

Explanation: NFR = quality attributes and constraints.

Q44. Domain requirements are often difficult because they may be:

- A) Always clear and explicit
- B) Written in business terms and implicit
- C) Purely technical
- D) Only about UI

✓ **Answer: B**

Explanation: Domain experts assume knowledge and may not specify details.

Q45. Correct order of Requirement Engineering steps is:

- A) Coding → Testing → Deployment → Elicitation
- B) Inception → Elicitation → Elaboration → Negotiation → Specification → Validation → Management
- C) Validation → Specification → Negotiation → Coding
- D) Modeling → Construction → Communication only

✅ **Answer: B**

Explanation: This is the standard RE workflow.

Q46. RTM (Requirement Traceability Matrix) is used to:

- A) Remove requirements
- B) Map requirements to design and test cases
- C) Replace SDLC model
- D) Encrypt requirement documents

✅ **Answer: B**

Explanation: RTM ensures traceability and coverage.

Q47. Software design mainly answers:

- A) WHAT the system must do
- B) HOW the system will be built internally
- C) WHO will use the system
- D) WHEN project ends

✅ **Answer: B**

Explanation: Analysis = WHAT, Design = HOW.

Q48. Which is NOT a design model element?

- A) Architectural design
- B) Interface design
- C) Component design
- D) Payroll processing

✅ **Answer: D**

Explanation: Payroll processing is a business function, not design element.

Q49. “High cohesion + Low coupling” indicates:

- A) Poor design
- B) Good design
- C) Always deadlock
- D) Only UI design

✅ **Answer: B**

Explanation: Golden rule of modular design.

Q50. FURPS stands for:

- A) Functionality, Usability, Reliability, Performance, Supportability
- B) Fast, Unique, Rapid, Powerful, Secure
- C) Functionality, UI, Runtime, Processing, Security
- D) None

✅ **Answer: A**

Explanation: Standard quality attribute model.

I) UML + OOAD

Q51. Use Case Diagram shows:

- A) Static object structure
- B) Actors and system interactions
- C) Database schema
- D) CPU scheduling

✅ **Answer: B**

Explanation: Use case = functional interactions from user perspective.

Q52. Class Diagram represents:

- A) Sequence of messages
- B) Workflow steps
- C) Static classes, attributes, methods, relationships
- D) Only user interface

✅ **Answer: C**

Explanation: Class diagram = static OOP structure.

Q53. Sequence Diagram explains:

- A) Order of interactions between objects over time
- B) System hardware layout
- C) Database normalization
- D) Network routing

✅ **Answer: A**

Explanation: Sequence shows time-ordered messages.

Q54. OOA (Object-Oriented Analysis) focuses on:

- A) HOW to code classes
- B) WHAT system must do
- C) Compiling code
- D) Deployment nodes

✅ **Answer: B**

Explanation: OOA = identifying actors/use-cases/objects.

Q55. OOD (Object-Oriented Design) focuses on:

- A) WHAT system must do
- B) HOW system will work internally
- C) Only requirement elicitation
- D) Only testing

✅ **Answer: B**

Explanation: OOD defines classes, methods, relationships, structure.

J) Coding Principles (SOLID + DRY + KISS)

Q56. SRP (Single Responsibility Principle) states:

- A) One class should have multiple responsibilities
- B) A class should have only one reason to change
- C) Classes should never change
- D) One method should do everything

✅ **Answer: B**

Explanation: SRP improves maintainability and clarity.

Q57. OCP (Open/Closed Principle) means:

- A) Open for modification, closed for extension
- B) Open for extension, closed for modification
- C) Closed-source software only
- D) No changes allowed ever

✅ **Answer: B**

Explanation: Extend behavior without changing existing stable code.

Q58. DRY means:

- A) Do Repeat Yourself
- B) Don't Repeat Yourself
- C) Don't Rewrite Yourself
- D) Do Reuse Yourself

✅ **Answer: B**

Explanation: Avoid duplicate code; keep reusable logic in one place.

Q59. KISS principle means:

- A) Keep It Simple
- B) Keep It Secure
- C) Keep It Separate
- D) Keep It Short

✅ **Answer: A**

Explanation: Prefer simple solutions over unnecessary complexity.

Q60. YAGNI means:

- A) You Always Get New Ideas
- B) You Aren't Gonna Need It
- C) You Are Good, Nice, Intelligent
- D) Your Application Gets New Inputs

✅ **Answer: B**

Explanation: Don't implement features until required.

Agile Software Development

Q1. Agile is best defined as:

- A) A rigid development model with fixed phases
- B) A lightweight, adaptive approach delivering software in small increments
- C) A testing-only methodology
- D) A hardware design process

✓ **Answer: B**

Explanation: Agile delivers usable increments frequently and adapts to change.

Q2. Agile delivery is typically:

- A) One final release at end
- B) Frequent small releases
- C) Only documentation delivered
- D) Release only after full system testing at end

✓ **Answer: B**

Explanation: Agile releases working software repeatedly.

Q3. In Agile, requirements are generally:

- A) Fixed early and never changed
- B) Evolving based on feedback
- C) Always ignored
- D) Only written by developers

✓ **Answer: B**

Explanation: Agile welcomes changing requirements.

Q4. Waterfall differs from Agile mainly because Waterfall:

- A) Welcomes requirement changes late
- B) Gives continuous feedback
- C) Has detailed upfront planning and fixed phases
- D) Delivers working software every week

✓ **Answer: C**

Explanation: Waterfall is plan-driven, sequential, and change is costly.

Q5. Testing in Agile is:

- A) Done only after coding phase
- B) Continuous and done in every sprint
- C) Not required
- D) Only manual testing at end

✓ **Answer: B**

Explanation: Agile integrates testing within sprint work.

Q6. Which is an Agile Manifesto value?

- A) Processes & tools over individuals
- B) Working software over comprehensive documentation

- C) Contract negotiation over customer collaboration
- D) Following plan over responding to change

✓ **Answer: B**

Explanation: Working software is valued more than heavy documentation.

Q7. Agile emphasizes:

- A) Responding to change over following a rigid plan
- B) No collaboration
- C) No customer feedback
- D) Only long-term planning

✓ **Answer: A**

Explanation: Agile adapts based on real feedback.

Q8. “Individuals and interactions” are valued over:

- A) Deadlocks and semaphores
- B) Processes and tools
- C) Architecture
- D) Deployment

✓ **Answer: B**

Explanation: Agile prioritizes people communication over rigid processes.

Q9. A Sprint/Iteration means:

- A) A long 2-year release cycle
- B) A fixed-length cycle delivering a usable increment
- C) A documentation phase
- D) A database design model

✓ **Answer: B**

Explanation: Sprint = short cycle producing working product.

Q10. Which is NOT a common Agile component?

- A) User stories
- B) Daily stand-ups
- C) Product backlog
- D) Belady’s anomaly

✓ **Answer: D**

Explanation: Belady’s anomaly belongs to OS paging, not Agile.

Q11. A key benefit of Agile is:

- A) Late validation only
- B) Faster delivery of value and early feedback
- C) No transparency
- D) Less collaboration

✓ **Answer: B**

Explanation: Agile reduces risk and improves stakeholder satisfaction.

Q12. Jira is mainly used for:

- A) CPU scheduling
- B) Agile project planning and tracking (backlog, sprint, issues)
- C) Page replacement
- D) Database indexing

✅ **Answer: B**

Explanation: Jira manages sprints, stories, boards, reports.

Q13. Which is a popular Agile tool?

- A) Jira
- B) BIOS
- C) GRUB
- D) GCC

✅ **Answer: A**

Explanation: Jira/Trello/Azure DevOps are Agile tools.

Q14. Scrum delivers work in:

- A) One big release
- B) Short fixed-length cycles called Sprints
- C) Only prototype
- D) Only design documents

✅ **Answer: B**

Explanation: Scrum is sprint-based.

Q15. Product Owner is responsible for:

- A) Writing all code
- B) Owning and prioritizing the Product Backlog
- C) Performing server installation
- D) Maintaining only test cases

✅ **Answer: B**

Explanation: PO sets priority based on business value.

Q16. Scrum Master mainly:

- A) Assigns tasks forcefully like a manager
- B) Facilitates Scrum, removes impediments, coaches the team
- C) Decides final business requirements alone
- D) Does database migration always

✅ **Answer: B**

Explanation: Scrum Master is a facilitator, not a boss.

Q17. Development Team in Scrum is:

- A) Only testers
- B) Cross-functional team delivering product increment
- C) Only managers
- D) Only UI designers

✅ **Answer: B**

Explanation: Dev team is cross-functional and builds increment.

Q18. Product Backlog is:

- A) List of bugs only
- B) Ordered list of all required features/enhancements
- C) Sprint daily report
- D) A test plan document

✅ **Answer: B**

Explanation: Product backlog evolves continuously.

Q19. Sprint Backlog is:

- A) Entire project backlog
- B) Subset of backlog selected for current Sprint
- C) Only deployment checklist
- D) Only customer feedback list

✅ **Answer: B**

Explanation: Sprint backlog = selected items + plan for sprint.

Q20. Increment means:

- A) Unfinished code
- B) Working tested usable output at end of Sprint
- C) Only documentation
- D) Only prototype UI

✅ **Answer: B**

Explanation: Increment must be potentially shippable.

Q21. Sprint Planning is used to:

- A) Merge branches in Git
- B) Define sprint goal and select backlog items
- C) Deploy to production directly
- D) Perform code compilation

✅ **Answer: B**

Explanation: Planning defines sprint objective + work selection.

Q22. Daily Scrum (Stand-up) is generally:

- A) 15-minute meeting to discuss progress + blockers
- B) 2-hour meeting for design
- C) Monthly meeting for release
- D) Only for Product Owner

✅ **Answer: A**

Explanation: Daily scrum syncs the team for next 24 hours.

Q23. Sprint Review is:

- A) Internal team improvement meeting
- B) Demo to stakeholders + feedback collection
- C) Only bug fixing session
- D) Only planning meeting

✅ **Answer: B**

Explanation: Review shows increment and gets feedback.

Q24. Sprint Retrospective focuses on:

- A) Product backlog prioritization
- B) Process improvement (what went well, what to improve)
- C) Writing new requirements only
- D) Only deployment automation

✅ **Answer: B**

Explanation: Retro improves team process continuously.

Q25. Sprint duration is typically:

- A) 1 day
- B) 1 to 4 weeks (fixed length)
- C) 6 months
- D) Undefined always

✅ **Answer: B**

Explanation: Scrum sprints are time-boxed.

Q26. XP mainly focuses on:

- A) Documentation only
- B) Technical excellence and high code quality
- C) No testing
- D) Only long releases

✅ **Answer: B**

Explanation: XP uses disciplined coding practices.

Q27. TDD means:

- A) Testing during deployment
- B) Test-Driven Development (write tests before code)
- C) Total Debugging Design
- D) Team Delivery Document

✅ **Answer: B**

Explanation: TDD = test first, then implement.

Q28. Pair Programming means:

- A) One developer works alone
- B) Two developers work together on one workstation
- C) Only managers write code
- D) Two projects run in parallel

✅ **Answer: B**

Explanation: One drives, one reviews continuously.

Q29. Refactoring means:

- A) Changing code behavior
- B) Improving code structure without changing external behavior
- C) Removing all tests
- D) Adding new requirements only

✅ **Answer: B**

Explanation: Refactoring improves design/maintainability safely.

Q30. Collective Code Ownership means:

- A) Only one person can modify code
- B) Any developer can improve any part of code
- C) Only Product Owner commits changes
- D) Code cannot be changed

✅ **Answer: B**

Explanation: XP encourages shared responsibility.

Q31. Kanban primarily uses:

- A) Gantt chart
- B) Visual workflow board (To Do → In Progress → Done)
- C) Spiral loops
- D) V-model mapping

✅ **Answer: B**

Explanation: Kanban is visual flow-based system.

Q32. The key goal of Kanban is:

- A) Increase Work-In-Progress always
- B) Limit WIP to improve flow and reduce overload
- C) Remove workflow stages
- D) Make sprints mandatory

✅ **Answer: B**

Explanation: WIP limits reduce bottlenecks.

Q33. Lean Software Development focuses on:

- A) Maximizing waste
- B) Waste reduction and value delivery
- C) Fixed requirements only
- D) No customer interaction

✅ **Answer: B**

Explanation: Lean removes non-value work to improve efficiency.

Q34. In Jira, a “Project” represents:

- A) Only a Git repository
- B) Workspace/container for a product or team work
- C) Only one sprint
- D) Only one user story

✅ **Answer: B**

Explanation: Jira project holds backlog, sprints, issues, reports.

Q35. In Jira, a “Story” represents:

- A) A defect report
- B) User-facing functionality/business requirement
- C) Server installation
- D) Code refactoring task always

✅ **Answer: B**

Explanation: Story = feature requirement from user view.

Q36. In Jira, an “Epic” is:

- A) A small task
- B) A large feature broken into multiple stories
- C) A sprint board
- D) A bug fix only

✅ **Answer: B**

Explanation: Epic groups related stories.

Q37. Burndown chart shows:

- A) Total CPU usage
- B) Remaining work vs time in a sprint
- C) Page faults vs frames
- D) Git commit history

✅ **Answer: B**

Explanation: Burndown tracks progress in sprint.

Q38. Velocity in Agile generally indicates:

- A) Bugs in system
- B) Work completed per sprint
- C) Number of stakeholders
- D) Code size only

✅ **Answer: B**

Explanation: Velocity helps forecast sprint capacity.

9) Agile Testing (Very High Weight)

Q39. In Agile, testing is:

- A) A separate phase after development
- B) Integrated throughout every sprint
- C) Skipped to save time
- D) Only performed by customer

✅ **Answer: B**

Explanation: Agile enforces continuous testing for stable increments.

Q40. Which testing is used to verify interactions between modules?

- A) Unit Testing
- B) Integration Testing
- C) System Testing
- D) UAT

✅ **Answer: B**

Explanation: Integration testing checks module-to-module connectivity.

Q41. UAT stands for:

- A) User Activity Tracking
- B) User Acceptance Testing
- C) Universal Agile Testing
- D) Unit Automation Test

✅ **Answer: B**

Explanation: UAT validates business needs from user/customer view.



DevOps & Containerization (Docker + Kubernetes)

Q1. DevOps primarily integrates:

- A) Hardware and networking only
- B) Development + Operations + Testing + Deployment/Monitoring
- C) Database and UI only
- D) OS and compiler only

✅ Answer: B

Explanation: DevOps unifies Dev, Ops, QA, Deploy, Monitor.

Q2. The main objective of DevOps is:

- A) Deliver software once per year
- B) Deliver high-quality software continuously and reliably
- C) Reduce testing completely
- D) Remove automation

✅ Answer: B

Explanation: DevOps focuses on continuous + stable delivery.

Q3. A key goal of DevOps is:

- A) Reduce deployment frequency
- B) Reduce time-to-market
- C) Increase manual work
- D) Avoid feedback

✅ Answer: B

Explanation: DevOps aims to deliver features faster.

Q4. DevOps removes:

- A) Testing and QA team
- B) Silos between development and operations teams
- C) Software requirements
- D) Version control

✅ Answer: B

Explanation: DevOps improves collaboration and shared ownership.

Q5. "Automation-first approach" in DevOps means:

- A) Manual testing only
- B) Automate build, test, deployment, monitoring, infrastructure
- C) Only automate coding
- D) Automate hardware manufacturing

✅ Answer: B

Explanation: DevOps heavily relies on automation across lifecycle.

Q6. IaC stands for:

- A) Infrastructure as Code
- B) Integration and Control

- C) Internet as Cloud
- D) Instance as Container

✅ **Answer: A**

Explanation: Infrastructure managed using code (repeatable setup).

Q7. “Fail-fast & recover quickly” means:

- A) Never deploy
- B) Encourage experimentation + fast rollback/recovery
- C) Ignore failures
- D) Stop monitoring

✅ **Answer: B**

Explanation: DevOps enables fast feedback and rapid fixes.

Q8. CI/CD in DevOps means:

- A) Code Installation / Code Deletion
- B) Continuous Integration / Continuous Delivery (or Deployment)
- C) Continuous Internet / Continuous Data
- D) Cloud Integration / Cloud Delivery

✅ **Answer: B**

Explanation: CI = merge+test often, CD = deliver frequently.

Q9. Correct DevOps lifecycle order includes:

- A) Monitor → Code → Plan → Deploy
- B) Plan → Code → Build → Test → Release → Deploy → Operate → Monitor
- C) Build → Deploy → Plan → Test
- D) Release → Plan → Monitor → Code

✅ **Answer: B**

Explanation: This is the standard continuous DevOps loop.

Q10. Which tool is commonly used for CI/CD automation?

- A) Jenkins
- B) BIOS
- C) GRUB
- D) FIFO

✅ **Answer: A**

Explanation: Jenkins / GitHub Actions automate pipelines.

Q11. Which tool is commonly used for monitoring in DevOps?

- A) Maven
- B) Prometheus + Grafana
- C) GitHub
- D) SVN

✅ **Answer: B**

Explanation: Prometheus collects metrics, Grafana visualizes.

Q12. A major deployment problem solved by containers is:

- A) Memory fragmentation
- B) Environment mismatch (works on my machine problem)
- C) Deadlock
- D) CPU scheduling

✅ **Answer: B**

Explanation: Containers package dependencies → consistent execution everywhere.

Q13. Containers are preferred over VMs mainly because they are:

- A) Larger and slower
- B) Lightweight and start quickly
- C) Require full OS per instance
- D) Cannot scale

✅ **Answer: B**

Explanation: Containers share host kernel → faster and smaller.

Q14. A container bundles:

- A) Only source code
- B) Only operating system
- C) Code + libraries + runtime dependencies + config
- D) Only database

✅ **Answer: C**

Explanation: Containers include everything needed to run the app.

Q15. Containers share:

- A) Host OS kernel
- B) Separate OS per container
- C) Separate hardware per container
- D) Separate BIOS per container

✅ **Answer: A**

Explanation: Containers use host kernel; VMs include guest OS.

Q16. Typical container size is:

- A) In GBs always
- B) Very small (MBs)
- C) Larger than VMs
- D) Always 1TB

✅ **Answer: B**

Explanation: Containers are lightweight vs VM images.

Q17. Startup time comparison:

- A) Containers = minutes, VMs = seconds
- B) Containers = seconds, VMs = minutes
- C) Both are same always
- D) Both take hours

✅ **Answer: B**

Explanation: Containers start faster than VMs.

Q18. Virtual machines provide:

- A) Process-level isolation only
- B) Full OS-level isolation
- C) No isolation
- D) Same as containers always

✅ **Answer: B**

Explanation: VM runs its own OS → full isolation.

Q19. Docker is best described as:

- A) A database engine
- B) A containerization platform to create/run containers
- C) A CPU scheduler
- D) A programming language

✅ **Answer: B**

Explanation: Docker standardizes packaging and execution.

Q20. Docker Engine is:

- A) A Dockerfile editor
- B) The runtime that executes and manages containers
- C) A cloud server
- D) Kubernetes service

✅ **Answer: B**

Explanation: Docker Engine is the core runtime.

Q21. Docker Image is:

- A) A running process
- B) A blueprint/template for creating containers
- C) A stopped container
- D) Only a config file

✅ **Answer: B**

Explanation: Image = template; Container = instance.

Q22. Docker Container is:

- A) A blueprint
- B) A running instance of an image
- C) A source code repository
- D) A VM always

✅ **Answer: B**

Explanation: Containers are created and run from images.

Q23. Dockerfile is:

- A) A binary executable
- B) A script of instructions to build an image
- C) A remote registry
- D) A monitoring tool

✅ **Answer: B**

Explanation: Dockerfile defines steps like FROM, COPY, RUN, CMD.

Q24. Docker Hub is:

- A) A local cache
- B) A public online image registry
- C) A Linux kernel module
- D) A database storage engine

✅ **Answer: B**

Explanation: Docker Hub hosts and shares images.

Q25. Docker Compose is used to:

- A) Run only one container
- B) Define and run multi-container applications
- C) Replace Kubernetes
- D) Replace Git

✅ **Answer: B**

Explanation: Compose runs app + DB + cache etc. together.

Q26. FROM in Dockerfile specifies:

- A) Container name
- B) Base image
- C) Port mapping
- D) Volume mount

✅ **Answer: B**

Explanation: FROM sets starting image (ubuntu/node/python).

Q27. COPY in Dockerfile is used to:

- A) Install packages
- B) Copy app files into image
- C) Start container
- D) Remove image

✅ **Answer: B**

Explanation: COPY adds source files into image build context.

Q28. RUN in Dockerfile is used for:

- A) Installing dependencies during build
- B) Running container at runtime only
- C) Creating Kubernetes pods
- D) Fetching Git commits

✅ **Answer: A**

Explanation: RUN executes commands while building image.

Q29. CMD in Dockerfile specifies:

- A) Base OS
- B) Default command executed when container starts
- C) Delete container
- D) Push image to hub

✅ **Answer: B**

Explanation: CMD runs at container runtime.

Q30. Correct workflow is:

- A) Container → Dockerfile → Image
- B) Dockerfile → Image → Container
- C) Image → Dockerfile → Container
- D) Dockerfile → Container → Image

✅ **Answer: B**

Explanation: Dockerfile builds image; image runs container.

Q31. Stopping a container will:

- A) Delete the image automatically
- B) Gracefully stop application inside container
- C) Delete Dockerfile
- D) Rebuild image automatically

✅ **Answer: B**

Explanation: docker stop stops container gracefully.

Q32. Destroying/removing a container means:

- A) Image is deleted automatically
- B) Container is deleted but image stays
- C) Container becomes a pod
- D) Docker Engine stops

✅ **Answer: B**

Explanation: Container removal doesn't delete the image.

Q33. Command to download image from Docker Hub:

- A) docker download
- B) docker fetch
- C) docker pull
- D) docker init

✅ **Answer: C**

Explanation: docker pull nginx.

Q34. Command to list images stored locally:

- A) docker list
- B) docker images
- C) docker ps
- D) docker view

✓ **Answer: B**

Explanation: Shows images present on system.

Q35. Command to build image from Dockerfile:

- A) docker make
- B) docker build
- C) docker compile
- D) docker create

✓ **Answer: B**

Explanation: Docker build creates an image.

Q36. Command to start a new container from image:

- A) docker run
- B) docker ps
- C) docker rmi
- D) docker diff

✓ **Answer: A**

Explanation: docker run creates + starts container.

Q37. Command to list running + stopped containers:

- A) docker ps -a
- B) docker list -l
- C) docker images -a
- D) docker pull -a

✓ **Answer: A**

Explanation: ps -a shows all containers.

Q38. Command to remove a stopped container:

- A) docker stop
- B) docker rm
- C) docker rmi
- D) docker kill

✓ **Answer: B**

Explanation: docker rm deletes container.

Q39. Command to remove an image:

- A) docker rm
- B) docker del
- C) docker rmi
- D) docker kill

✅ **Answer: C**

Explanation: docker rmi removes image.

Q40. Container orchestration is needed because:

- A) One container cannot run
- B) Managing hundreds/thousands of containers manually is difficult
- C) Docker cannot run containers
- D) Git cannot store code

✅ **Answer: B**

Explanation: Large scale needs auto-management.

Q41. Kubernetes provides:

- A) Load balancing, auto-scaling, self-healing, rolling updates
- B) Only file compression
- C) Only CPU scheduling
- D) Only database replication

✅ **Answer: A**

Explanation: Kubernetes handles large-scale container ops.

Q42. Kubernetes configurations are generally written in:

- A) Java
- B) YAML
- C) HTML
- D) SQL

✅ **Answer: B**

Explanation: YAML defines resources and desired state.

Q43. metadata in Kubernetes YAML contains:

- A) Resource name and labels
- B) CPU scheduling algorithm
- C) Git history
- D) Dockerfile instructions

✅ **Answer: A**

Explanation: metadata identifies resource.

Q44. spec in Kubernetes YAML contains:

- A) Only resource history
- B) Configuration like replicas, image, ports
- C) Only error logs
- D) Passwords only

✅ **Answer: B**

Explanation: spec defines desired settings.

Q45. IaC means:

- A) Infrastructure changes managed manually
- B) Infrastructure configurations stored and version-controlled like code
- C) Only UI code
- D) Only testing scripts

✅ **Answer: B**

Explanation: Infrastructure is repeatable and consistent.

Q46. Docker Swarm is:

- A) Docker's native clustering and orchestration tool
- B) A database engine
- C) A Linux file system
- D) A CI pipeline tool

✅ **Answer: A**

Explanation: Swarm manages Docker clusters and services.

Q47. Docker Stack is used to:

- A) Deploy multi-service applications on Swarm
- B) Replace Docker Engine
- C) Replace Kubernetes pods
- D) Remove images

✅ **Answer: A**

Explanation: Stack deploys multi-service apps across Swarm nodes.

Q48. Kubernetes is:

- A) A programming language
- B) Open-source container orchestration platform
- C) A database
- D) A browser

✅ **Answer: B**

Explanation: Kubernetes deploys, scales, manages container apps.

Q49. A Kubernetes Cluster is:

- A) A single container
- B) A group of machines (nodes) working together
- C) Only a YAML file
- D) Only a docker image

✅ **Answer: B**

Explanation: Cluster has multiple nodes.

Q50. A Node in Kubernetes is:

- A) Only the master node
- B) A machine (VM/physical) that runs workloads
- C) A docker registry
- D) A Git branch

✅ **Answer: B**

Explanation: Node is a compute machine in the cluster.

Q51. The Control Plane (Master Node) is responsible for:

- A) Running user application containers directly always
- B) Managing scheduling and orchestration of cluster
- C) Storing docker images
- D) Only monitoring logs

✅ **Answer: B**

Explanation: Control plane manages state, scheduling, orchestration.

Q52. Worker Node's main job is:

- A) Manage cluster decisions
- B) Execute application workloads (pods)
- C) Store Git commits
- D) Build docker images only

✅ **Answer: B**

Explanation: Worker nodes run actual application containers.

Q53. The smallest deployable unit in Kubernetes is:

- A) Deployment
- B) Service
- C) Pod
- D) Cluster

✅ **Answer: C**

Explanation: Pod is smallest unit, usually contains 1 container.

Q54. Kubernetes Service provides:

- A) A stable network endpoint to access Pods
- B) A compiler for containers
- C) An OS kernel module
- D) Code review

✅ **Answer: A**

Explanation: Service keeps stable access even if pods restart.

Q55. Kubernetes Deployment manages:

- A) Only Docker Hub images
- B) Pods, replicas, and rolling updates
- C) Only logs
- D) Only sprint planning

✅ **Answer: B**

Explanation: Deployment ensures desired replicas and rolling updates.

Q56. Kubernetes Deployment YAML typically defines:

- A) Only Git branches
- B) App image + replicas + ports
- C) Only employee details
- D) Only testing reports

✅ **Answer: B**

Explanation: YAML config includes application runtime details.

Q57. Kubernetes “Self-healing” means:

- A) It writes application code automatically
- B) It restarts failed containers/pods automatically
- C) It deletes all pods
- D) It disables scaling

✅ **Answer: B**

Explanation: Failed pods are recreated automatically.

Q58. Kubernetes “Rolling Update” means:

- A) Deploy everything at once with downtime
- B) Deploy new version gradually without downtime
- C) Stop entire cluster permanently
- D) Only update documentation

✅ **Answer: B**

Explanation: Rolling updates replace pods gradually.

Q59. Kubernetes Dashboard is used for:

- A) Managing Git commits
- B) Visual monitoring and management of cluster resources
- C) Writing Dockerfiles
- D) Compiling YAML

✅ **Answer: B**

Explanation: Dashboard shows workloads, logs, nodes, scaling.

Software Testing & Integration

Q1. Software testing is mainly done to:

- A) Prove software has zero defects
- B) Execute software with intention to find defects
- C) Replace software development
- D) Increase hardware speed

✅ **Answer: B**

Explanation: Testing finds defects; it cannot prove “no defects”.

Q2. Testing is:

- A) Only performed after deployment
- B) Only a documentation task
- C) A planned and systematic activity to evaluate quality
- D) Optional in SDLC

✅ **Answer: C**

Explanation: Testing is structured and happens throughout SDLC.

Q3. The best reason why testing is critical is:

- A) Bugs are cheaper to fix in production
- B) Early defect detection reduces rework cost
- C) Testing removes all risks completely
- D) Testing is only for UI

✅ **Answer: B**

Explanation: Early bugs cost less than production failures.

Q4. Verification answers:

- A) Are we building the right product?
- B) Are we building the product right?
- C) Is system fast?
- D) Is database normalized?

✅ **Answer: B**

Explanation: Verification checks correctness vs specifications.

Q5. Validation answers:

- A) Are we building the product right?
- B) Are we building the right product?
- C) Is coding style correct?
- D) Is compiler working?

✅ **Answer: B**

Explanation: Validation ensures meeting user needs.

Q6. Code inspection is an example of:

- A) Validation
- B) Verification

- C) UAT
- D) Load testing

✓ **Answer: B**

Explanation: Inspection/review = verification (static).

Q7. User Acceptance Testing (UAT) is an example of:

- A) Verification
- B) Validation
- C) Static testing
- D) Compiler test

✓ **Answer: B**

Explanation: UAT validates real user requirements.

Q8. Quality Assurance (QA) is:

- A) Product-oriented
- B) Process-oriented and preventive
- C) Only bug fixing
- D) Only execution of test cases

✓ **Answer: B**

Explanation: QA ensures correct process to prevent defects.

Q9. Quality Control (QC) is:

- A) Process improvement only
- B) Product inspection and evaluation
- C) Requirement gathering
- D) Project planning only

✓ **Answer: B**

Explanation: QC checks actual output/product quality.

Q10. Testing is mainly a part of:

- A) QA
- B) QC
- C) Project management
- D) Deployment pipeline only

✓ **Answer: B**

Explanation: Testing detects defects in product → QC activity.

Q11. Which is a correct testing principle?

- A) Testing proves absence of defects
- B) Exhaustive testing is possible
- C) Testing reveals defects, not prove zero defects
- D) Testing is not context dependent

✓ **Answer: C**

Explanation: Testing shows defects; can't guarantee perfection.

Q12. "Exhaustive testing is impossible" means:

- A) Testing is useless
- B) All inputs and paths cannot be tested practically
- C) Only developers can test
- D) No need of test cases

☒ **Answer: B**

Explanation: Testing must be risk-based due to limitations.

Q13. Defects usually cluster in:

- A) All modules equally
- B) Few critical/high-risk modules
- C) Only UI modules
- D) Only database layer

☒ **Answer: B**

Explanation: Most defects occur in small set of modules.

Q14. "Absence of defects does not guarantee usefulness" means:

- A) Software is useless always
- B) Even defect-free system may not satisfy user needs
- C) Testing is optional
- D) Requirements are not important

☒ **Answer: B**

Explanation: If wrong product built, it can still fail business goals.

Q15. Requirement Analysis in STLC is used to:

- A) Execute test cases
- B) Identify testable requirements and scope
- C) Close testing process
- D) Deploy to production

☒ **Answer: B**

Explanation: Understand scope and acceptance criteria.

Q16. Test Planning phase produces:

- A) Test execution report
- B) Test Plan document
- C) Defect report
- D) Code module

☒ **Answer: B**

Explanation: Planning defines strategy, schedule, tools, roles.

Q17. Test Design phase includes:

- A) Creating test cases and expected results
- B) Deploying software
- C) Creating Docker containers
- D) Writing code modules

✓ **Answer: A**

Explanation: Design prepares scenarios, test data, expected output.

Q18. Environment Setup is performed to:

- A) Fix defects
- B) Prepare stable testing environment (hardware/OS/tools/build)
- C) Merge branches
- D) Estimate effort

✓ **Answer: B**

Explanation: Execution requires stable test environment.

Q19. Defect Reporting & Tracking includes:

- A) Logging defects + retesting + regression testing
- B) Only requirement gathering
- C) Only writing code
- D) Only planning

✓ **Answer: A**

Explanation: Defects are tracked until closure and retesting is done.

Q20. Test Closure includes:

- A) Writing new requirements
- B) Archiving test artifacts + lessons learned
- C) Code deployment only
- D) Creating UI

✓ **Answer: B**

Explanation: Closure formally ends testing with reports/analysis.

Q21. In V-Model, Requirement Analysis corresponds to:

- A) Unit testing
- B) Acceptance testing
- C) Integration testing
- D) Smoke testing

✓ **Answer: B**

Explanation: Requirements ↔ Acceptance testing.

Q22. In V-Model, System Design corresponds to:

- A) System testing
- B) Unit testing
- C) UAT
- D) Bug fixing

✓ **Answer: A**

Explanation: System design ↔ System testing.

Q23. In V-Model, Module Design corresponds to:

- A) Acceptance testing
- B) Unit testing (planned)
- C) Load testing
- D) Alpha testing

✅ **Answer: B**

Explanation: Module/component design ↔ Unit testing.

Q24. Unit testing checks:

- A) Entire system at once
- B) Individual functions/modules in isolation
- C) User satisfaction
- D) Production performance only

✅ **Answer: B**

Explanation: Unit tests smallest components.

Q25. Unit testing is mainly done by:

- A) Customers
- B) Developers
- C) Business analysts
- D) Network admins

✅ **Answer: B**

Explanation: Developers test code-level units.

Q26. Integration testing focuses on:

- A) Code formatting
- B) Interaction and interfaces between modules
- C) Only UI colors
- D) Only requirement writing

✅ **Answer: B**

Explanation: Modules may pass unit tests but fail when combined.

Q27. System testing verifies:

- A) Only one function
- B) Entire system end-to-end
- C) Only database indexes
- D) Only class diagrams

✅ **Answer: B**

Explanation: System testing validates complete system behavior.

Q28. UAT is performed by:

- A) Developers only
- B) End-users/business representatives
- C) Compiler only
- D) CI/CD tool only

✓ **Answer: B**

Explanation: UAT is final approval before go-live.

Q29. A stub is used to:

- A) Simulate calling module
- B) Simulate called module
- C) Replace database
- D) Replace requirements

✓ **Answer: B**

Explanation: Stub acts as fake lower-level module called by higher module.

Q30. A driver is used to:

- A) Simulate called module
- B) Simulate calling module
- C) Fix bugs
- D) Deploy software

✓ **Answer: B**

Explanation: Driver acts as fake caller for lower-level module.

Q31. Top-down integration means:

- A) Integrate from lower modules to higher modules
- B) Integrate from higher modules to lower modules
- C) Integrate randomly
- D) Integrate only UI first

✓ **Answer: B**

Explanation: Starts from top modules and moves downward.

Q32. Bottom-up integration means:

- A) Integrate from lowest modules upward
- B) Start with UI only
- C) Start with acceptance testing
- D) Skip unit testing

✓ **Answer: A**

Explanation: Low-level modules integrated first.

Q33. Sandwich/Hybrid integration is:

- A) Only top-down
- B) Only bottom-up
- C) Combination of top-down and bottom-up
- D) No integration testing needed

✓ **Answer: C**

Explanation: Uses both approaches to reduce issues.

Q34. Regression testing ensures:

- A) Old functionality is not affected by new changes
- B) Only new feature works
- C) Only UI is correct
- D) Database backup is done

✓ **Answer: A**

Explanation: Regression confirms “nothing broke”.

Q35. Smoke testing is:

- A) Deep detailed testing
- B) Quick basic build verification test
- C) Only performance testing
- D) Only stress testing

✓ **Answer: B**

Explanation: Smoke checks build stability for further testing.

Q36. White-box testing focuses on:

- A) Requirements without code knowledge
- B) Internal logic and code paths
- C) UI usability only
- D) Business workflow only

✓ **Answer: B**

Explanation: White-box = structural testing.

Q37. Black-box testing focuses on:

- A) Internal code structure
- B) External behavior based on requirements
- C) Compiler optimization
- D) Database indexing

✓ **Answer: B**

Explanation: Black-box ignores internal implementation.

Q38. Grey-box testing means:

- A) No knowledge of system
- B) Partial knowledge of internal system
- C) Full code access only
- D) Only UI testing

✓ **Answer: B**

Explanation: Combines black-box + white-box.

Q39. Functional testing ensures:

- A) System meets performance targets only
- B) System performs required functions correctly
- C) System uses minimal memory
- D) System uses best algorithm

✓ **Answer: B**

Explanation: Functional = feature correctness.

Q40. Non-functional testing evaluates:

- A) Only login functionality
- B) Quality attributes like performance, security, usability
- C) Only reports generation
- D) Only file creation

✓ **Answer: B**

Explanation: NFR testing checks quality characteristics.

Q41. Equivalence Partitioning means:

- A) Testing all input values
- B) Dividing inputs into valid/invalid groups and testing one from each
- C) Testing only boundary values
- D) Testing randomly

✓ **Answer: B**

Explanation: One representative from each partition is sufficient.

Q42. Boundary Value Analysis focuses on:

- A) Middle values only
- B) Values near boundaries (min/max and just outside)
- C) Only invalid values
- D) Only performance values

✓ **Answer: B**

Explanation: Most bugs occur at boundaries.

Q43. Error guessing is:

- A) Formal mathematical method only
- B) Experience-based technique to predict defects
- C) Only automation testing
- D) Only unit testing

✓ **Answer: B**

Explanation: Tester uses intuition for likely failure areas.

Q44. Population-based testing uses:

- A) Random data always
- B) Realistic production-like data
- C) Only boundary values
- D) Only invalid inputs

✓ **Answer: B**

Explanation: Real-world patterns reveal real issues.

Q45. Test Plan includes:

- A) Only test execution results
- B) Scope, strategy, environment, roles, risks, schedule
- C) Only bug list
- D) Only code flow

✅ **Answer: B**

Explanation: Test plan organizes and controls testing process.

Q46. A good test case must have:

- A) No expected result
- B) Clear steps and measurable expected result
- C) Only test title
- D) Only developer notes

✅ **Answer: B**

Explanation: Expected result must be specific and testable.

Q47. Integration testing is critical because:

- A) Modules never fail together
- B) Modules may work alone but fail when combined
- C) It replaces unit testing
- D) It is only for UI

✅ **Answer: B**

Explanation: Interface mismatch/data flow errors appear during integration.

Q48. Typically, testing effort in projects is around:

- A) 5–10%
- B) 10–15%
- C) 30–40%
- D) 90–95%

✅ **Answer: C**

Explanation: Testing takes major effort and improves product quality.

Automation Testing with Selenium

Q1. Selenium is mainly used for:

- A) Database testing
- B) Web application automation testing
- C) Desktop application testing
- D) OS process scheduling

✅ **Answer: B**

Explanation: Selenium automates browsers for web apps only.

Q2. Selenium is:

- A) Paid and closed-source
- B) Free and open-source
- C) Only available for Windows
- D) Only for Chrome browser

✅ **Answer: B**

Explanation: Selenium is open-source and cross-platform.

Q3. Selenium supports:

- A) Only Java
- B) Only Python
- C) Multiple programming languages like Java, Python, C#, JS
- D) Only C language

✅ **Answer: C**

Explanation: Selenium works with many languages.

Q4. Selenium can automate:

- A) Desktop software like MS Word
- B) Web applications through browsers
- C) Mobile apps directly without tools
- D) Performance testing

✅ **Answer: B**

Explanation: Selenium is for web apps only.

Q5. Selenium integrates well with CI/CD tools like:

- A) Jenkins
- B) GCC
- C) BIOS
- D) GRUB

✅ **Answer: A**

Explanation: Selenium runs in pipelines using Jenkins/GitHub Actions.

Q6. The main Selenium component used in real projects is:

- A) Selenium IDE
- B) Selenium Grid

- C) Selenium WebDriver
- D) Selenium Compiler

✅ **Answer: C**

Explanation: WebDriver is the core API for browser automation.

Q7. Selenium IDE is mainly used for:

- A) Cross-browser parallel execution
- B) Record and playback for learning/prototyping
- C) Performance testing
- D) API automation testing

✅ **Answer: B**

Explanation: IDE is for quick recording, not large frameworks.

Q8. Selenium Grid is used for:

- A) Database backup
- B) Running tests on multiple browsers/machines in parallel
- C) Writing test scripts automatically
- D) Converting manual tests into unit tests

✅ **Answer: B**

Explanation: Grid enables distributed parallel execution.

Q9. To run Selenium WebDriver tests you need:

- A) Only browser
- B) Selenium libraries + browser driver + language runtime
- C) Only Selenium IDE
- D) Only Docker

✅ **Answer: B**

Explanation: WebDriver needs libraries + driver + runtime.

Q10. Chrome browser automation requires:

- A) GeckoDriver
- B) SafariDriver
- C) ChromeDriver
- D) EdgeDriver only

✅ **Answer: C**

Explanation: ChromeDriver is used for Chrome.

Q11. Correct communication flow in Selenium is:

- A) Browser → WebDriver → Driver
- B) WebDriver → Browser Driver → Browser
- C) Browser Driver → WebDriver → OS
- D) WebDriver → OS → Browser only

✅ **Answer: B**

Explanation: WebDriver sends commands to driver, driver controls browser.

Q12. "Selense" refers to:

- A) OS scheduling commands
- B) Commands to automate browser actions
- C) SQL commands
- D) Linux shell scripting commands

☒ **Answer: B**

Explanation: Selenium commands simulate user actions.

Q13. Selenium can automate:

- A) Clicking, typing, selecting dropdown, verifying output
- B) CPU allocation
- C) Disk formatting
- D) Kernel execution

☒ **Answer: A**

Explanation: Selenium automates UI interactions.

Q14. Locators are used to:

- A) Store test reports
- B) Find and identify web elements
- C) Compile Java code
- D) Optimize database queries

☒ **Answer: B**

Explanation: Locators tell Selenium which element to act on.

Q15. Which is NOT a Selenium locator?

- A) id
- B) name
- C) xpath
- D) compiler

☒ **Answer: D**

Explanation: Compiler is not a locator.

Q16. Which locator is usually fastest and most reliable if available?

- A) XPath
- B) ID
- C) Partial link text
- D) Tag name

☒ **Answer: B**

Explanation: ID is unique and stable (best choice).

Q17. XPath is especially useful for:

- A) Only static pages
- B) Dynamic or complex elements
- C) API testing
- D) Database schema

✅ **Answer: B**

Explanation: XPath can locate elements using structure, text, attributes.

Q18. Absolute XPath is generally:

- A) Highly stable and recommended
- B) Fragile and not recommended
- C) Faster than ID always
- D) Used only in Linux

✅ **Answer: B**

Explanation: Absolute path breaks easily when UI changes.

Q19. Relative XPath is:

- A) Longer and fragile
- B) Shorter and more stable
- C) Always invalid
- D) Same as absolute XPath

✅ **Answer: B**

Explanation: Relative XPath is preferred for maintainable scripts.

Q20. CSS Selector locator works using:

- A) OS scheduling rules
- B) CSS rules to locate elements
- C) SQL queries
- D) Java bytecode

✅ **Answer: B**

Explanation: CSS Selector targets HTML elements with CSS patterns.

Q21. Radio buttons allow selection of:

- A) Multiple options
- B) Only one option in a group
- C) No options
- D) Unlimited text

✅ **Answer: B**

Explanation: Radio = single selection from group.

Q22. Checkboxes allow:

- A) Only one option
- B) One or more options
- C) Only numeric input
- D) Only links

✅ **Answer: B**

Explanation: Checkbox supports multi-selection.

Q23. A dropdown selection in Selenium can be done by:

- A) Visible text, value, index
- B) Only index
- C) Only mouse hover
- D) Only using XPath

✅ **Answer: A**

Explanation: Dropdown selection can be done by text/value/index.

Q24. Keyboard & mouse advanced actions (hover, drag-drop) are handled by:

- A) WebDriver Alerts API
- B) WebDriver Actions API
- C) Selenium Grid
- D) Docker Engine

✅ **Answer: B**

Explanation: Actions API supports complex user interactions.

Q25. Multi-select element testing should verify:

- A) Only one value can be selected
- B) Multiple selections and deselection behavior
- C) Only default value
- D) Only browser title

✅ **Answer: B**

Explanation: Multi-select must support multiple choices + removal.

Q26. Selenium automation is best suited for:

- A) Regression testing and repeated execution
- B) One-time testing only
- C) Kernel testing
- D) Hardware testing

✅ **Answer: A**

Explanation: Automation gives best ROI in repeatable tests.

Q27. Selenium is NOT ideal for:

- A) Regression testing
- B) Smoke testing
- C) Highly unstable and frequently changing UI
- D) Cross-browser validation

✅ **Answer: C**

Explanation: Dynamic unstable UI breaks scripts frequently.

Q28. “Golden Rule” of Selenium automation is:

- A) Automation replaces manual testing completely
- B) Automation supports manual testing, not replaces it
- C) Manual testing is useless
- D) Only automation should be done

✅ **Answer: B**

Explanation: Manual is still needed for exploratory/visual validations.

Q29. A test suite is:

- A) A single test case
- B) A group of test cases executed together
- C) Only bug reports
- D) Only Selenium Grid setup

✅ **Answer: B**

Explanation: Suite allows batch execution.

Q30. Good test suite design should include:

- A) Dependent test cases always
- B) Reusable functions, setup/teardown, logging, reporting
- C) No exception handling
- D) Only UI clicks

✅ **Answer: B**

Explanation: Robust framework = reusable + maintainable + clear reports.

Q31. Popular automation frameworks in Java include:

- A) TestNG and JUnit
- B) Flask and Django
- C) React and Angular
- D) Kubernetes and Docker

✅ **Answer: A**

Explanation: TestNG/JUnit are testing frameworks for Java automation.

Q32. Popular Python automation frameworks include:

- A) PyTest and unittest
- B) Maven and Gradle
- C) SVN and CVS
- D) Jira and Trello

✅ **Answer: A**

Explanation: PyTest/unittest commonly used for Selenium in Python.

Q33. Test suites help in:

- A) CI/CD pipeline execution and scheduled runs
- B) OS booting
- C) Page replacement
- D) Memory compaction

✅ **Answer: A**

Explanation: Suites are executed in batch, parallel, scheduled in pipelines.

Q34. One major benefit of Selenium is:

- A) Only supports one browser
- B) Cross-browser testing capability
- C) Only supports desktop apps
- D) Only supports database testing

✅ **Answer: B**

Explanation: Selenium runs on Chrome/Firefox/Edge/Safari.

Q35. Selenium provides high ROI mainly for:

- A) Regression testing
- B) One-time manual checks
- C) CPU testing
- D) Data recovery

✅ **Answer: A**

Explanation: Repeated regression tests benefit most.

Q36. Cloud platforms like BrowserStack/Sauce Labs help with:

- A) Kernel compilation
- B) Remote testing across browsers/devices
- C) Git branching
- D) Memory management

✅ **Answer: B**

Explanation: They provide cross-device browser environments.

Q37. Selenium cannot directly automate:

- A) Web applications
- B) Desktop applications
- C) Login forms
- D) Dropdown selection

✅ **Answer: B**

Explanation: Selenium is for web UI only.

Q38. Selenium cannot perform:

- A) Performance testing
- B) UI automation testing
- C) Regression automation
- D) Cross-browser testing

✅ **Answer: A**

Explanation: Selenium is not a performance testing tool.

Q39. For mobile automation testing (instead of Selenium), commonly used tool is:

- A) Appium
- B) Jenkins
- C) Maven
- D) Jira

✅ **Answer: A**

Explanation: Appium is used for mobile automation.

Q40. Selenium requires strong focus on:

- A) Stable element locators and test maintenance
- B) Removing all test frameworks
- C) Avoiding scripting knowledge
- D) Avoiding CI/CD

✅ **Answer: A**

Explanation: Locators + maintenance decide success of automation.



CI/CD & Jenkins Integration

Q1. CI/CD primarily automates:

- A) Only writing code
- B) The entire software delivery process (build, test, deploy, feedback)
- C) Only UI design
- D) Only database indexing

✅ **Answer: B**

Explanation: CI/CD automates build → test → package → deploy → feedback.

Q2. A Delivery Pipeline is:

- A) Manual process of delivering software
- B) A series of automated steps from code commit to deployment
- C) Only a testing document
- D) A database workflow

✅ **Answer: B**

Explanation: Pipeline acts like a production line for software delivery.

Q3. Typical pipeline starts with:

- A) Deployment
- B) Monitoring
- C) Source code commit (Git)
- D) Production rollback

✅ **Answer: C**

Explanation: Pipeline begins when code is pushed to Git.

Q4. Which stage comes immediately after source code commit in pipeline?

- A) Build
- B) Monitoring
- C) UAT
- D) Release approval

✅ **Answer: A**

Explanation: After commit, system builds and compiles the project.

Q5. Main goal of CI/CD is:

- A) Slower releases
- B) Faster, repeatable and reliable software delivery
- C) Only documentation generation
- D) Increasing manual work

✅ **Answer: B**

Explanation: CI/CD makes delivery fast and consistent.

Q6. Continuous Integration means:

- A) Developers commit code rarely
- B) Developers frequently commit to shared repo and build/tests run automatically

- C) Manual testing only
- D) Deploy directly without tests

✅ **Answer: B**

Explanation: CI integrates code often and checks early.

Q7. One major benefit of CI is:

- A) More merge conflicts
- B) Defects found early with fast feedback
- C) No need of Git
- D) Code becomes unstable

✅ **Answer: B**

Explanation: CI prevents “big bang” integration issues.

Q8. CI is best remembered as:

- A) Continuous Installation
- B) Continuously integrated and tested code
- C) Continuous Internet
- D) Continuous Infrastructure

✅ **Answer: B**

Explanation: CI keeps codebase working.

Q9. Continuous Delivery means:

- A) Every successful build auto deploys to production
- B) Software is always in ready-to-deploy state (production deploy may be manual approval)
- C) No automated testing
- D) Only packaging is automated

✅ **Answer: B**

Explanation: Delivery = ready anytime, but deploy can be manual.

Q10. Continuous Deployment means:

- A) Manual deployment after approval
- B) Every build that passes tests deploys to production automatically
- C) No CI required
- D) Only code compilation

✅ **Answer: B**

Explanation: Deployment = fully automatic release to production.

Q11. Key difference between Continuous Delivery and Continuous Deployment is:

- A) CI is missing in both
- B) Continuous deployment removes human approval for production release
- C) Continuous delivery is slower than manual always
- D) Continuous deployment does not use testing

✅ **Answer: B**

Explanation: Delivery may require approval, deployment does not.

Q12. CI/CD improves quality mainly due to:

- A) Removing testing
- B) Continuous automated testing and fast feedback
- C) Running only manual tests
- D) Skipping builds

✅ **Answer: B**

Explanation: Automated tests detect defects early.

Q13. CI/CD reduces risk because:

- A) Releases are larger and rare
- B) Small frequent releases are easier to manage and rollback
- C) No monitoring is needed
- D) Code is never merged

✅ **Answer: B**

Explanation: Smaller release = less blast radius.

Q14. Jenkins is:

- A) A database tool
- B) A CI/CD automation server
- C) A programming language
- D) A container runtime

✅ **Answer: B**

Explanation: Jenkins automates build/test/deploy workflows.

Q15. Jenkins is popular mainly because it has:

- A) No plugins
- B) Huge plugin ecosystem + integrations
- C) Works only with SVN
- D) No UI support

✅ **Answer: B**

Explanation: Plugins make Jenkins powerful and flexible.

Q16. Jenkins can integrate with:

- A) Git, Maven, Docker, Kubernetes, Selenium
- B) Only Windows OS
- C) Only Chrome browser
- D) Only HTML projects

✅ **Answer: A**

Explanation: Jenkins supports most DevOps toolchain.

Q17. Jenkins is managed primarily using:

- A) BIOS screen
- B) Web browser dashboard
- C) Linux kernel
- D) Only command line

✓ **Answer: B**

Explanation: Jenkins UI dashboard controls jobs/pipelines.

Q18. Jenkins Agent (Slave Node) is used to:

- A) Store Git history
- B) Execute jobs/builds on additional machines
- C) Write Jenkinsfile automatically
- D) Replace Jenkins server

✓ **Answer: B**

Explanation: Agents distribute workload and improve performance.

Q19. Main reason to use Jenkins nodes/agents is:

- A) Slow down pipeline
- B) Share workload + support multiple OS builds
- C) Remove CI/CD
- D) Replace testing frameworks

✓ **Answer: B**

Explanation: Agents enable parallel builds and multi-platform support.

Q20. Jenkins agents can be:

- A) Only Linux servers
- B) Only Windows servers
- C) Linux/Windows/Cloud/Docker containers
- D) Only physical servers

✓ **Answer: C**

Explanation: Agents can be anywhere: VM, cloud, containers.

Q21. A Jenkins Pipeline is:

- A) A manual checklist
- B) Automated workflow defined in Jenkins
- C) Only a Git branch
- D) Only a Selenium script

✓ **Answer: B**

Explanation: Pipeline automates stages like build/test/deploy.

Q22. Freestyle Job in Jenkins is:

- A) Always written in Jenkinsfile
- B) GUI-based job setup (simple tasks)
- C) Only for Kubernetes
- D) Only for monitoring

✓ **Answer: B**

Explanation: Freestyle = GUI approach, easiest.

Q23. Pipeline-as-Code in Jenkins is written in:

- A) Dockerfile
- B) Jenkinsfile
- C) package.json
- D) pom.xml

✅ **Answer: B**

Explanation: Jenkins pipelines are stored as Jenkinsfile.

Q24. Advantage of Pipeline-as-Code is:

- A) Cannot be version controlled
- B) Stored in version control, reusable, portable
- C) Only works offline
- D) No visibility in code review

✅ **Answer: B**

Explanation: Jenkinsfile is tracked like source code.

Q25. Typical Jenkins pipeline stages include:

- A) Checkout → Build → Test → Package → Deploy → Notify
- B) Only deploy
- C) Only commit
- D) Only monitoring

✅ **Answer: A**

Explanation: This is standard CI/CD pipeline design.

Q26. Why integrate Selenium with Jenkins?

- A) To disable testing
- B) To run automated UI/regression tests continuously in pipeline
- C) To replace Git
- D) To make deployment slower

✅ **Answer: B**

Explanation: Selenium automation ensures build quality continuously.

Q27. Correct Selenium CI flow is:

- A) Run tests → write code → deploy
- B) Checkout tests → install deps → run tests → generate reports → publish results
- C) Deploy first → test later
- D) Test only once yearly

✅ **Answer: B**

Explanation: Pipeline must execute and report automatically.

Q28. If Selenium tests fail in pipeline, Jenkins should:

- A) Mark build as failed and notify team
- B) Ignore failure and deploy anyway
- C) Delete code
- D) Remove Jenkins agents

✅ **Answer: A**

Explanation: Fail fast prevents bad deployments.

Q29. A best practice for CI/CD is:

- A) Commit code rarely
- B) Commit frequently and keep main branch stable
- C) Avoid automation
- D) Skip smoke tests

✅ **Answer: B**

Explanation: Frequent commits reduce conflicts and improve stability.

Q30. Pipeline should stop immediately when:

- A) Everything passes
- B) A stage fails (fail-fast)
- C) Monitoring is enabled
- D) Code is committed

✅ **Answer: B**

Explanation: Fail-fast prevents waste and broken deployments.

Q31. Correct strategy in pipelines is:

- A) Run heavy regression tests first always
- B) Run quick smoke tests first, deep regression later
- C) Skip unit tests
- D) Never run integration tests

✅ **Answer: B**

Explanation: Smoke tests quickly validate build stability.

Q32. Security best practice in Jenkins/CI-CD includes:

- A) Publicly store passwords in code
- B) Role-based access and encrypted credentials
- C) Disable authentication
- D) Run pipeline without logs

✅ **Answer: B**

Explanation: Credentials must be secured, access controlled.

Q33. CI/CD is considered a part of DevOps because it:

- A) Reduces automation
- B) Enables fast, repeatable, quality-driven delivery
- C) Removes collaboration
- D) Works only in waterfall

✅ **Answer: B**

Explanation: CI/CD is core to DevOps continuous delivery.

Q34. Jenkins acts as:

- A) Database server
- B) Central orchestrator (“brain”) of CI/CD workflow
- C) Web browser
- D) API gateway only

☒ **Answer: B**

Explanation: Jenkins connects tools and runs pipeline stages.

